

LAPORAN TUGAS MODUL PRAKTIKUM 3
MATA KULIAH PENGEMBANGAN APLIKASI MOBILE



OLEH:
HARIS ALFARISY
NIM.245150401111009

DOSEN PENGAMPU:
Muhammad Aminul Akbar, S.Kom., M.T.

SISTEM INFORMASI
DEPARTEMEN SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
2026

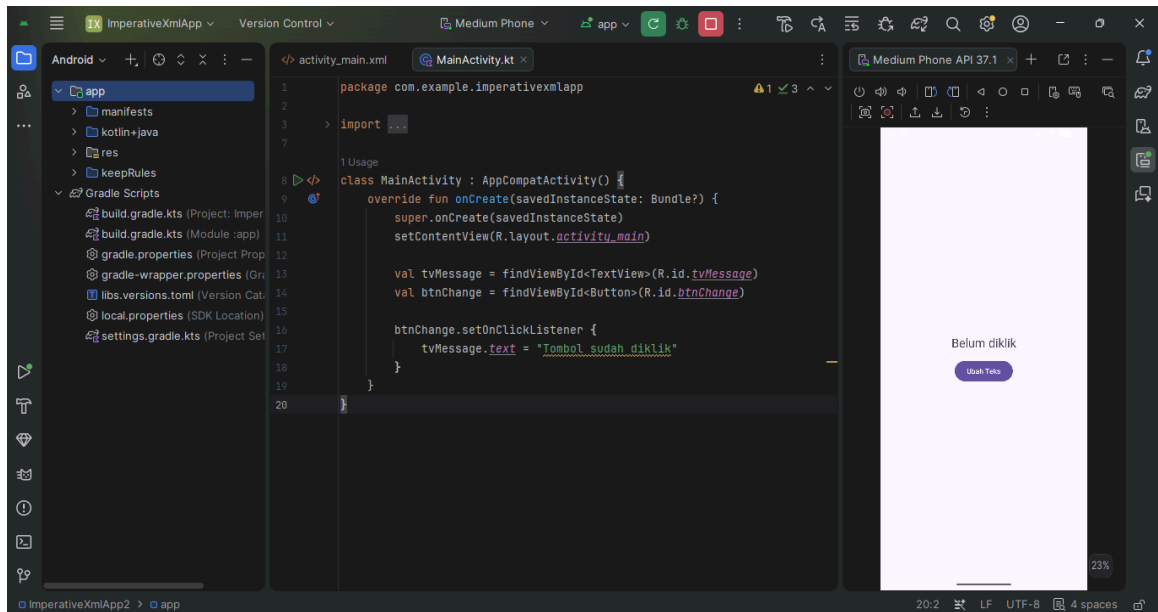
A. Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Aktivitas Percobaan

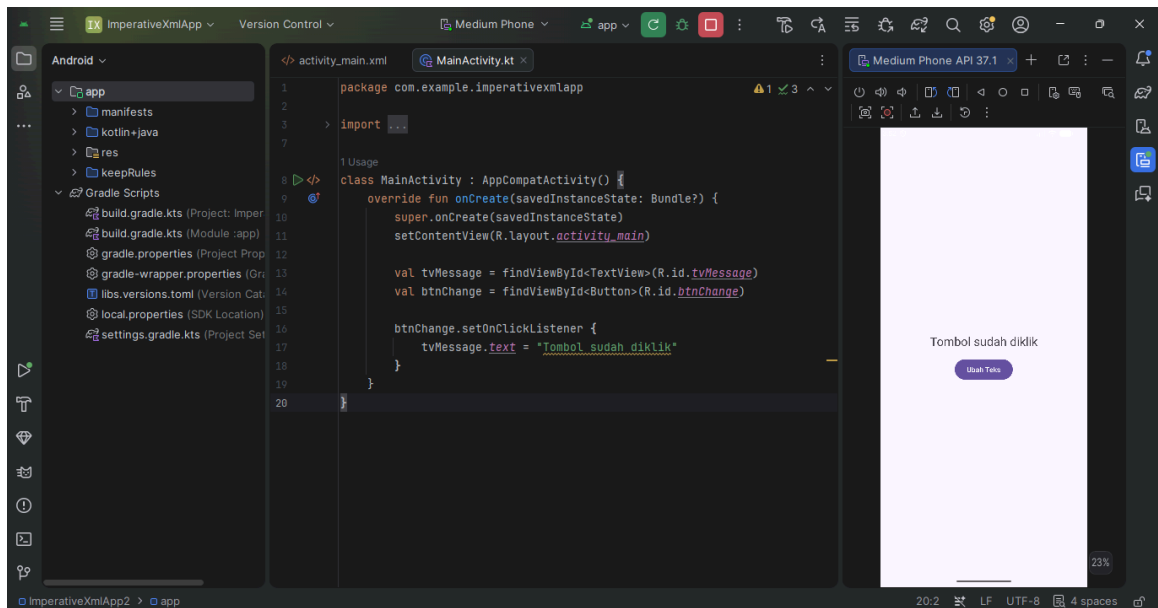
1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Project A - XML/View (Imperatif)

Aplikasi sebelum di klik:

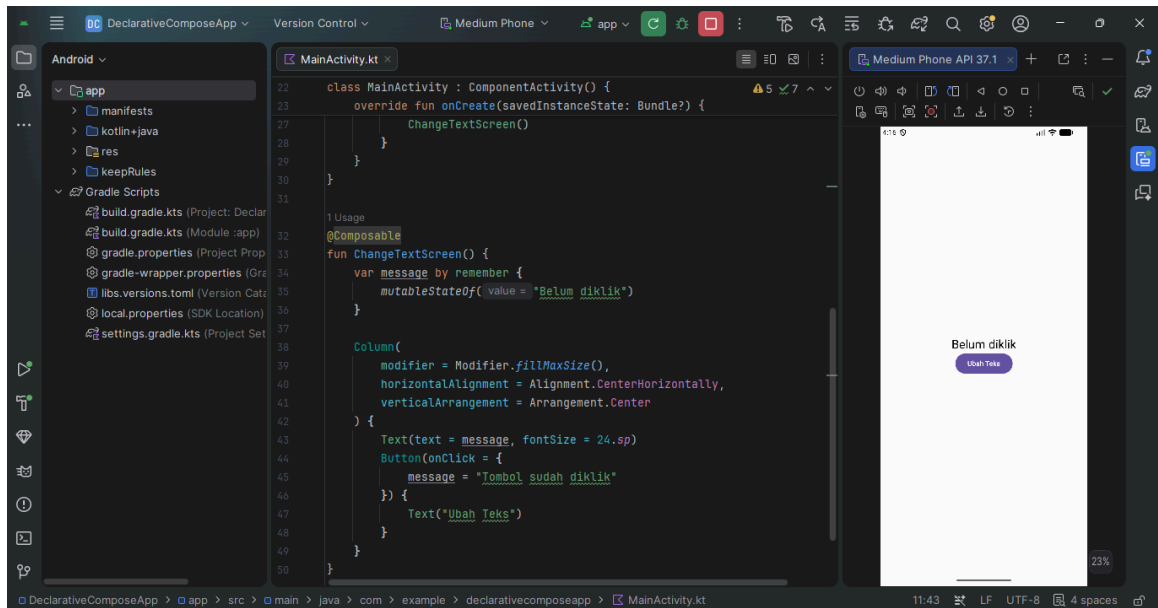


Aplikasi sesudah di klik, teks berubah:

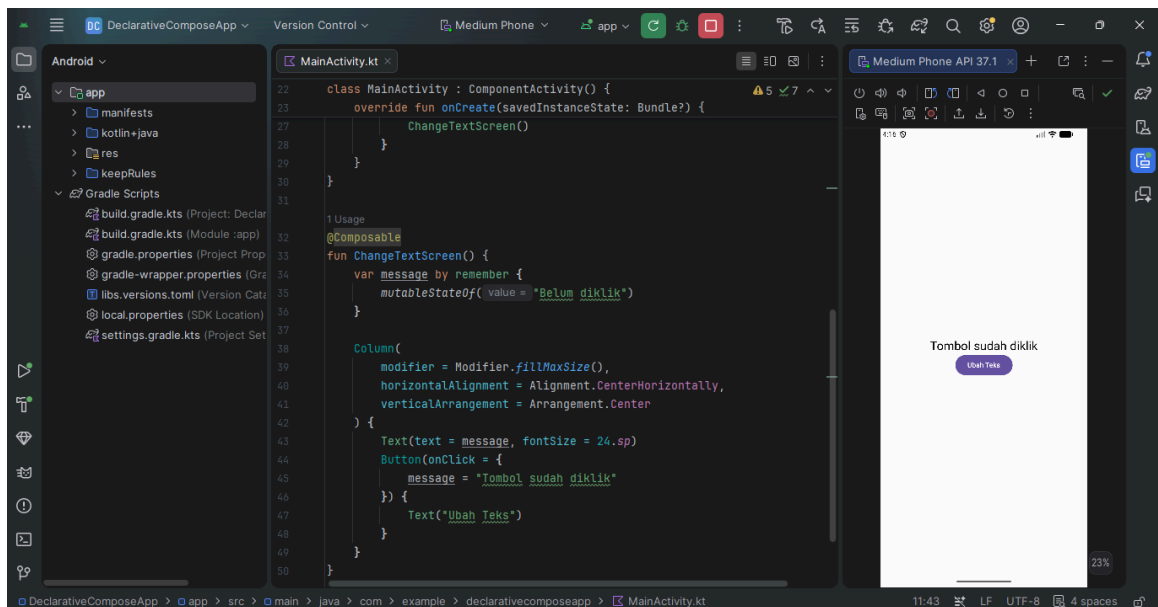


Project B - Jetpack Compose (Deklaratif)

Aplikasi sebelum di klik:



Aplikasi sesudah di klik, teks berubah:



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawab: objek yang diubah secara langsung ketika tombol di klik yaitu properti text yang ada pada objek TextView (tvMessage)

Baris kodenya: `tvMessage.text = "Tombol sudah diklik"`

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawab: nilai yang berubah ketika tombol di klik yaitu pada bagian variabel message

Baris kodenya: `message = "Tombol sudah diklik"`

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Jawab: karena pada jetpack compose menerapkan paradigma deklaratif. Tampilannya tidak dimanipulasi secara langsung lewat referensi objek view. UI dideklarasikan secara langsung pada state. Ketika state berubah, maka compose secara otomatis memperbarui tampilan lewat mekanisme recomposition

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawab: alurnya yaitu dimulai dari event (klik tombol) lalu ke view mencari objek textview melalui referensi dan lanjut ke perubahan tampilan dengan mengubah properti `tvMessage.text` secara eksplisit dan langsung

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawab: sistem alurnya yaitu dimulai dari event melakukan klik maka di bagian state akan memperbarui nilai message nya, lalu lanjut recomposition dimana compose akan mendeteksi perubahan state lalu merender ulang composable yang bergantung padanya dan terakhir lanjut pada perubahan tampilan yang akan menampilkan text nilai state yang baru

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jawab: maka yang akan terjadi yaitu variabel lokal biasa tidak dapat diamati oleh compose karena compose membutuhkan tipe state yang reaktif untuk mengetahui kapan harus memicu untuk proses recomposition

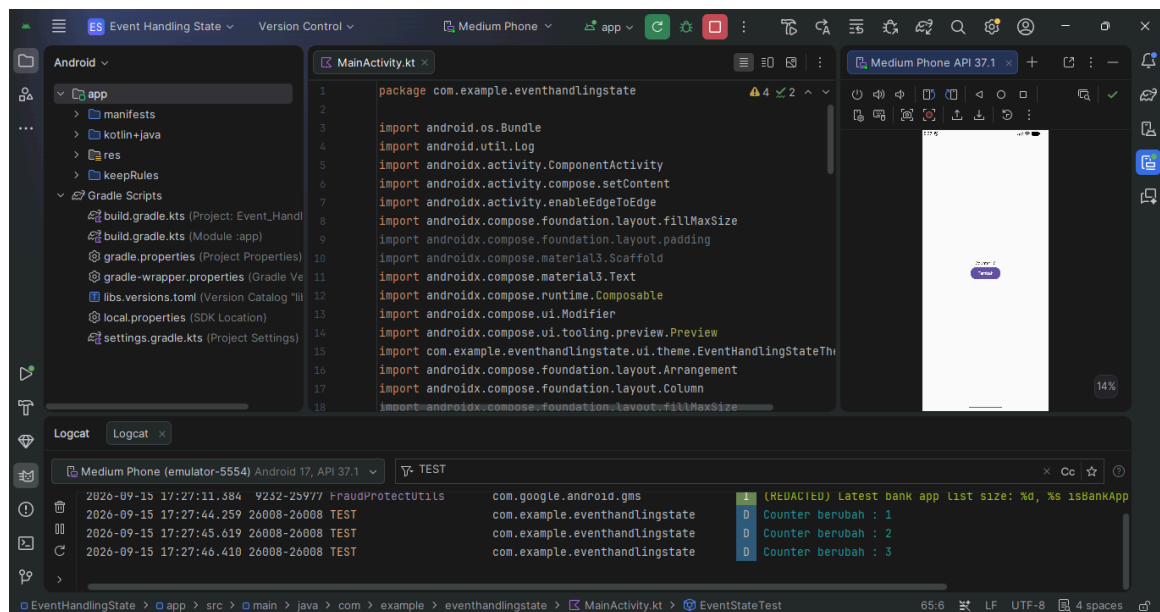
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Jawab: Berpikir secara imperatif yaitu berpikir dengan berfokus pada langkah demi langkah untuk mengambil elemen antarmuka, lalu menginstruksikan perubahan propertinya secara langsung, seperti "bagaimana cara mengubahnya". Berpikir secara deklaratif yaitu berpikir dengan fokus ke pendefinisian UI nya sebagai fungsi dari state, developer hanya bertugas untuk memperbarui state yang nantinya framework yang mengurus penggambaran ulang antarmuka, seperti "seperti apa tampilannya berdasarkan kondisi saat ini"

B. Event Handling State

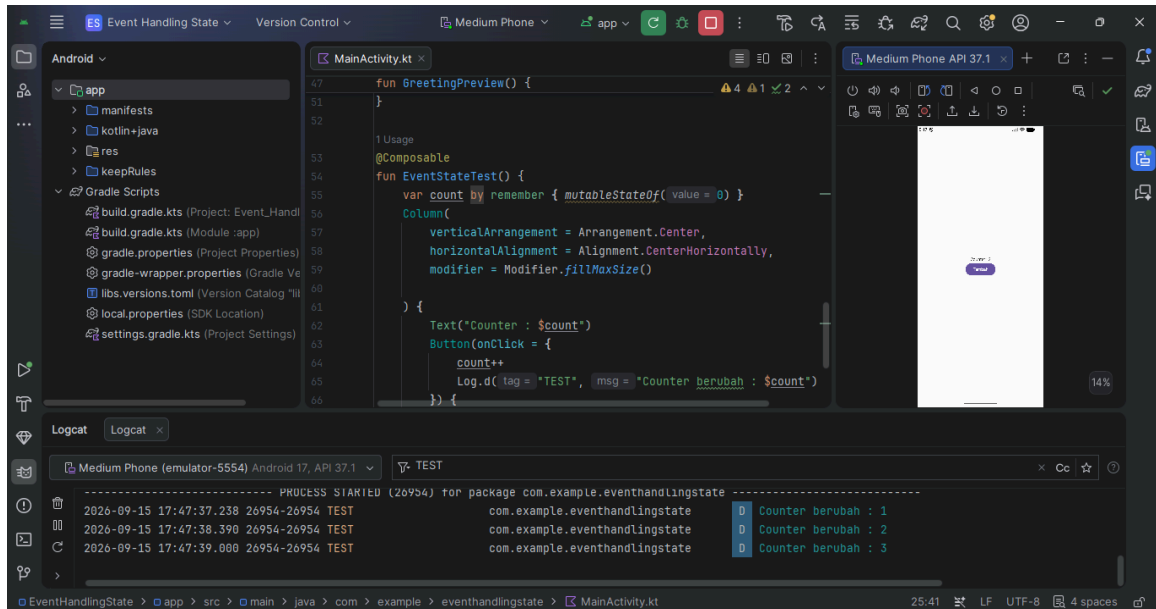
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" di klik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" di klik?
Jawab: yang muncul di logcat yaitu pesan log bertambah nilai secara beruntun, seperti TEST: Counter berubah 1, dan seterusnya
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" di klik?
Jawab: pada tampilan aplikasi ketika tombol di klik, teks Counter : 0 tidak mengalami perubahan atau penambahan angka
4. Apa yang menyebabkan hal tersebut terjadi?
Jawab: hal tersebut terjadi dikarenakan count dideklarasikan sebagai variabel biasa, bukan state yang dapat diamati. Nilai count sebenarnya bertambah di memori tetapi di compose tidak menganggapnya sebagai perubahan state yang memicu recomposition. Akibatnya, UI tidak pernah ditampilkan ulang dan tetap seperti tampilan awal
5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.
6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya
7. Berikut kode lengkap EventStateTest

8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab: yang muncul di logcat yaitu pesan log bertambah nilai secara beruntun, seperti TEST: Counter berubah 1, dan seterusnya

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab: tampilan pada aplikasi berubah, teks counter selalu bertambah jika tombol di klik

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

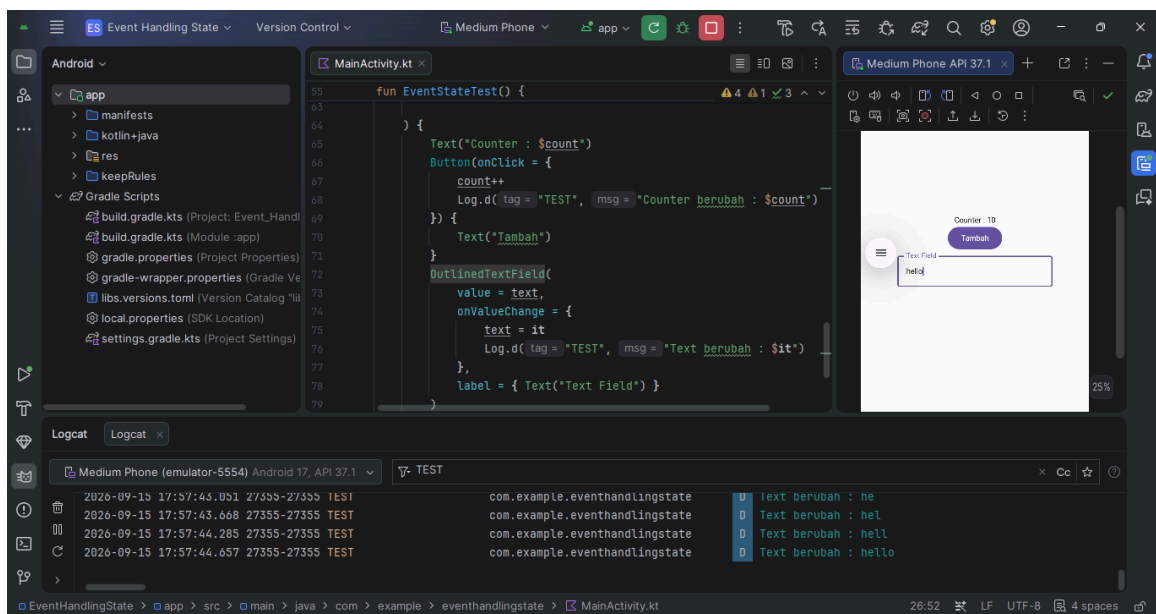
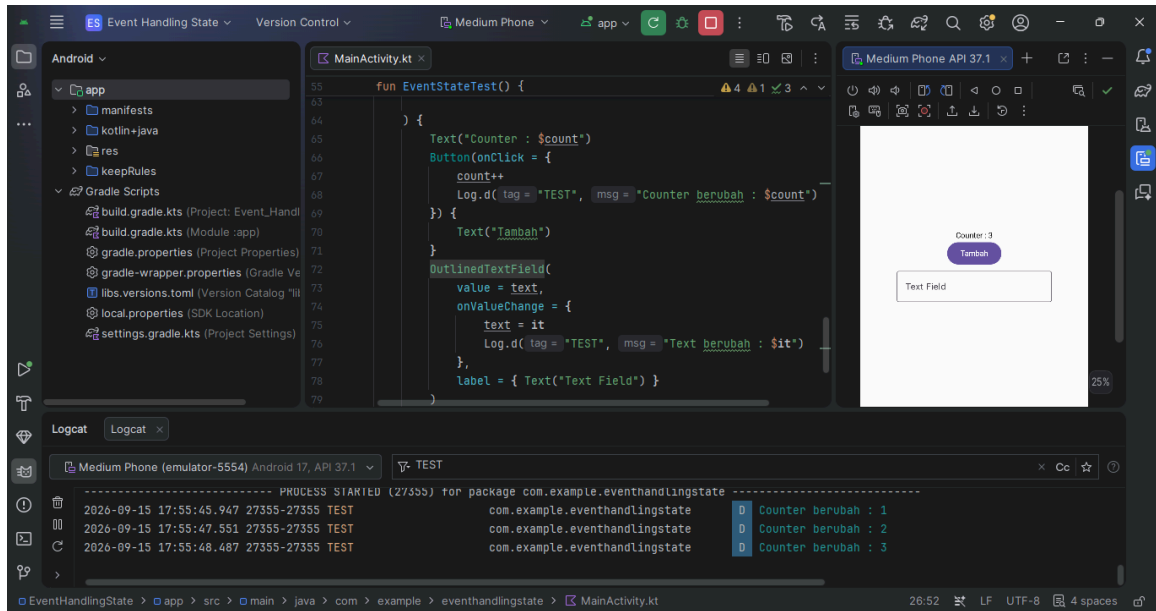
Jawab: hal tersebut terjadi dikarenakan count sekarang dideklarasikan sebagai state menggunakan mutableStateOf dan disimpan dengan remember. Ketika nilai state berubah, Compose mendeteksi perubahan tersebut dan menjalankan recomposition, sehingga UI diperbarui. Pada percobaan sebelumnya, variabel biasa tidak memicu recomposition.

12. Apa yang dimaksud recomposition pada Compose?

Jawab: recomposition adalah proses di mana compose menjalankan ulang fungsi composable yang membaca state yang berubah, untuk memperbarui UI agar sesuai dengan state terbaru.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" di klik beberapa kali.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawab: setiap karakter yang diketik memicu onChange dan mencatat teks terkini

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawab: teks yang diketik langsung muncul di OutlinedTextField. UI diperbarui secara real-time mengikuti input pengguna

17. Apa yang menyebabkan hal tersebut terjadi?

Jawab: hal tersebut terjadi karena setiap kali user mengetik, `onValueChange` dipanggil, state text diperbarui, dan `compose` melakukan recomposition sehingga `OutlinedTextField` menampilkan teks terbaru

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Jawab: state yang ada di program yaitu `count` yang bertipe integer, menyimpan jumlah counter dan `text` yang bertipe string, menyimpan teks input dari `TextField`

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Jawab: event yang ada di dalam program yaitu `onClik` yang berfungsi sebagai button untuk tambah dan `onValueChange` yang ada di `OutlinedTextField`

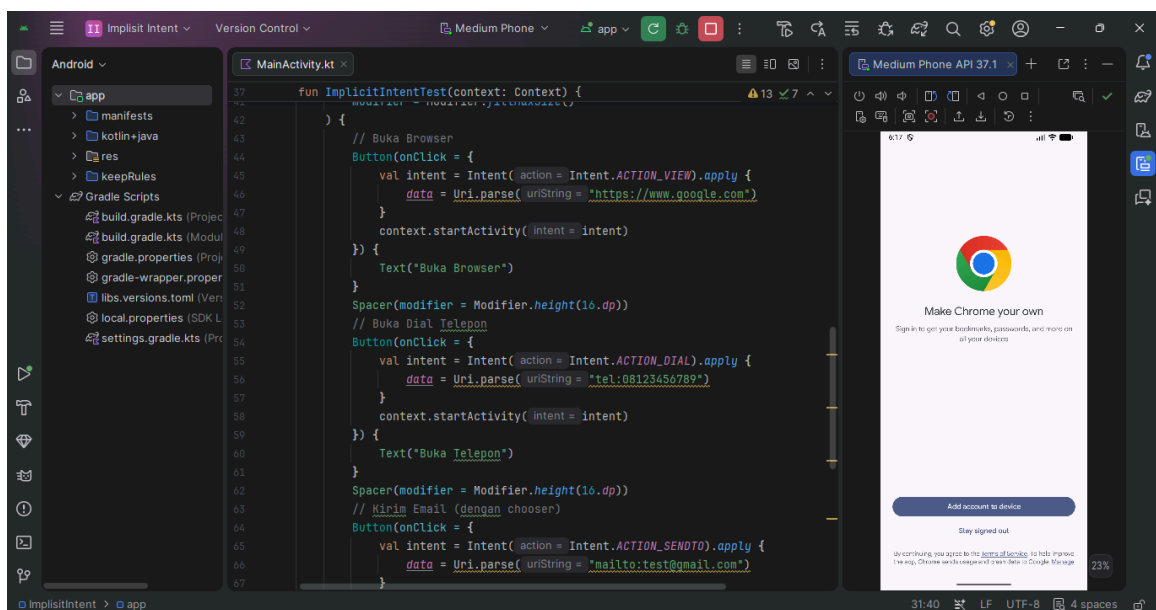
20. Kapan event - event tersebut dijalankan?

Jawab: event `onClick` dijalankan ketika user menekan tombol "Tambah" dan event `onValueChange` akan dijalankan ketika user mengetik atau mengubah text di dalam `TextField`

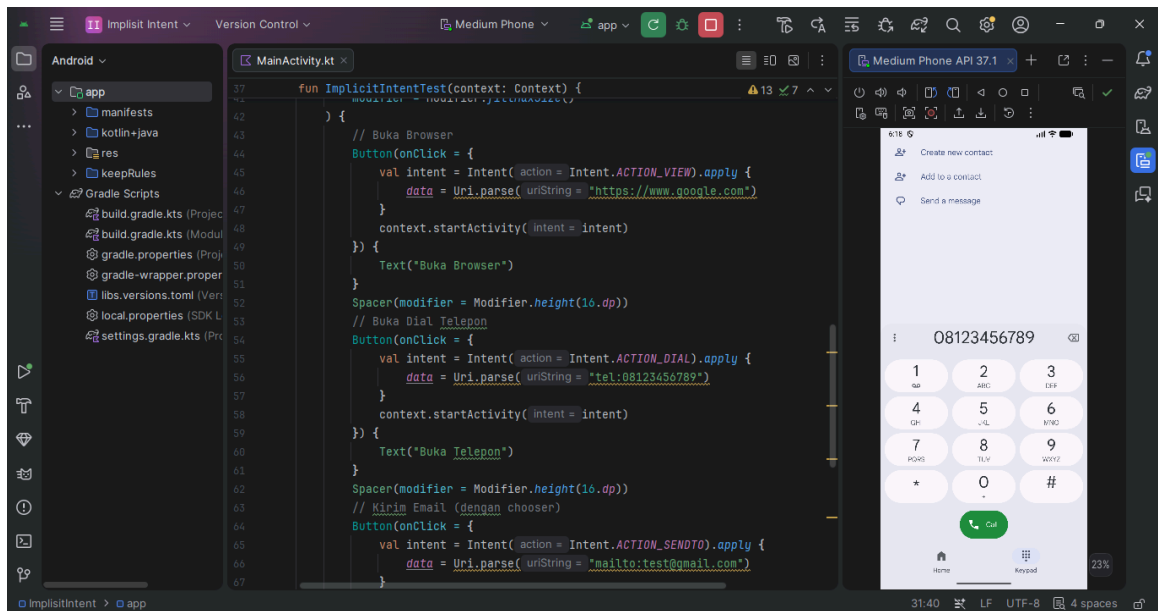
C. Implisit Intent

Aktivitas Percobaan

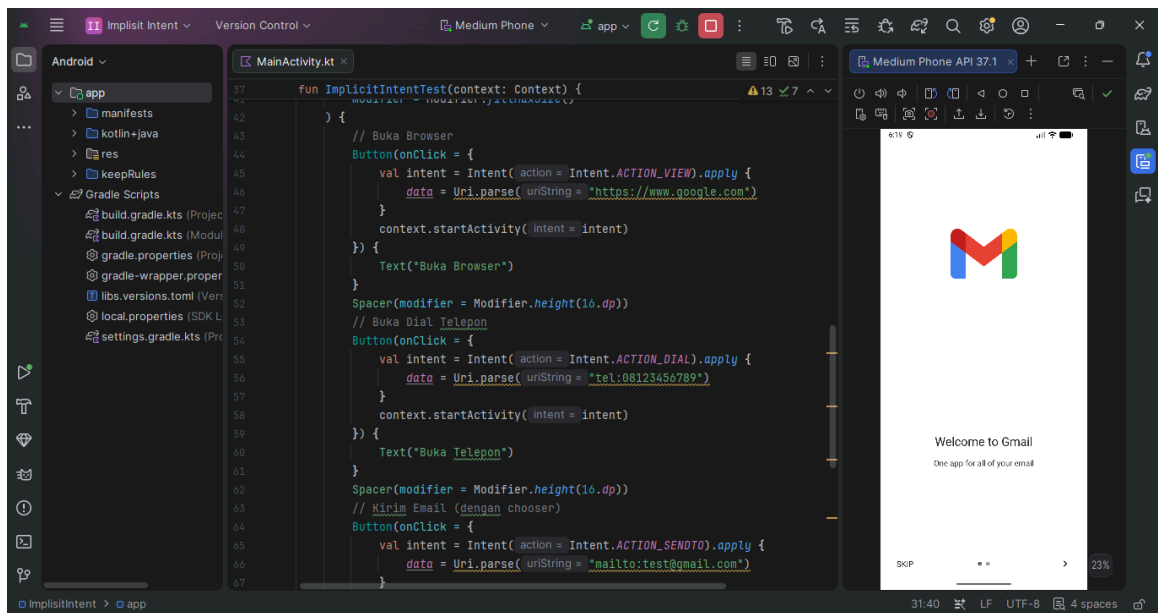
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:
2. Klik Buka Browser



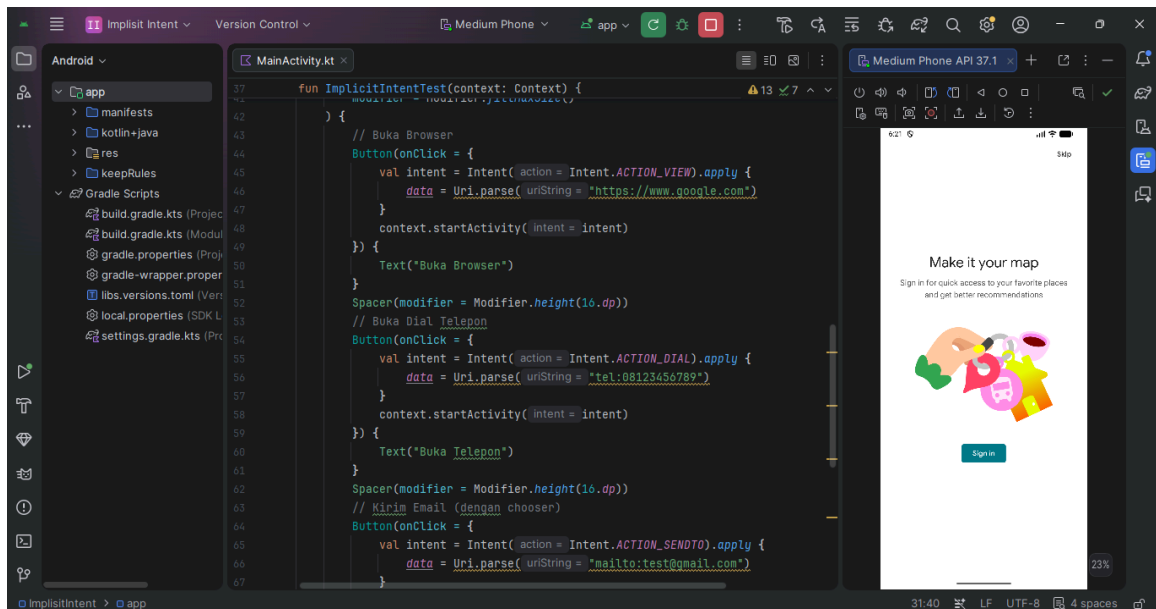
3. Klik Buka Telepon



4. Klik Kirim Email



5. Klik Buka Maps



6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya (di atas)

7. Pada kode yang digunakan, terdapat beberapa action seperti:

8. `Intent.ACTION_VIEW`

9. `Intent.ACTION_DIAL`

10. `Intent.ACTION_SENDTO`

11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

`Intent.ACTION_VIEW` digunakan untuk menampilkan data kepada pengguna, contohnya membuka URL di browser, membuka lokasi di maps. Sistem akan memilih aplikasi yang bisa menampilkan data tersebut.

`Intent.ACTION_DIAL` digunakan untuk membuka aplikasi telepon dengan nomor yang sudah terisi di dialer, tetapi tidak langsung untuk menelpon. Pengguna masih harus menekan tombol panggil untuk menelpon.

`Intent.ACTION_SENDTO` digunakan untuk mengirim data ke tujuan tertentu, biasanya untuk email atau SMS.

12. Pada kode yang digunakan, terdapat beberapa data seperti:

13. `data = Uri.parse("mailto:test@gmail.com")`

14. `data = Uri.parse("https://www.google.com")`

15. `data = Uri.parse("geo:-7.250445,112.768845")`

16. Apa fungsi data pada intent ?

Jawab: data pada intents berfungsi sebagai informasi yang akan menjadi target atau resource yang akan diproses oleh action. Data akan menentukan apa yang akan dibuka nantinya.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Jawab: action menentukan jenis operasi yang akan dilakukan, sedangkan data menentukan objek atau target dari operasi tersebut. Sistem android mencocokkan kombinasi action dan data dengan intent filter yang dideklarasikan oleh aplikasi yang terpasang. Aplikasi yang memiliki intent filter yang cocok akan muncul di dialog pemilihan aplikasi atau chooser.